

Experiment-4

Cript-Arithmetic Problem

Aim:

To develop a Python program to solve the Cryptarithmic Problem using Backtracking.

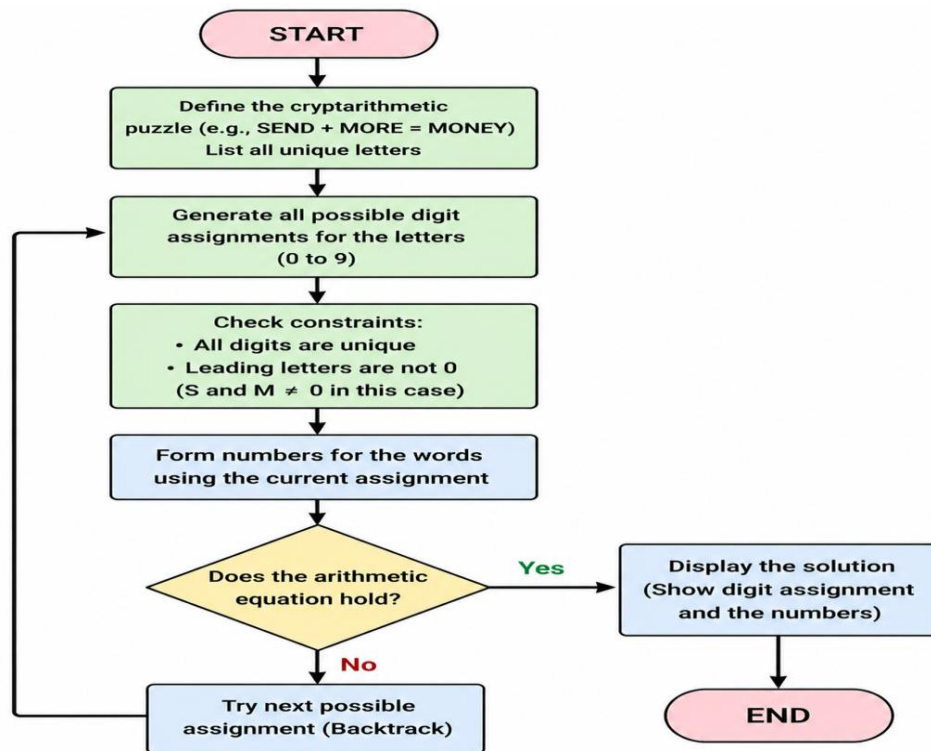
Objective:

- To solve a cryptarithmic puzzle using Backtracking.
- To assign unique digits to letters while satisfying the arithmetic equation.
- To understand constraint satisfaction techniques in Artificial Intelligence.

Algorithm:

1. Start the program.
2. Define the letters in the cryptarithmic puzzle.
3. Generate all possible digit assignments.
4. Ensure each letter has a unique digit.
5. Do not allow leading letters to be zero.
6. Form the numbers represented by the words.
7. Check whether the arithmetic equation is satisfied.
8. If valid, display the solution.
9. Stop the program.

Flowchart:



Code:

```
from itertools import permutations
```

```
letters = ('S','E','N','D','M','O','R','Y')
```

```
for p in permutations(range(10), 8):
```

```
    d = dict(zip(letters, p))
```

```
    if d['S']==0 or d['M']==0:
```

```
        continue
```

```
    send = d['S']*1000+d['E']*100+d['N']*10+d['D']
```

```
    more = d['M']*1000+d['O']*100+d['R']*10+d['E']
```

```
    money = d['M']*10000+d['O']*1000+d['N']*100+d['E']*10+d['Y']
```

```
    if send + more == money:
```

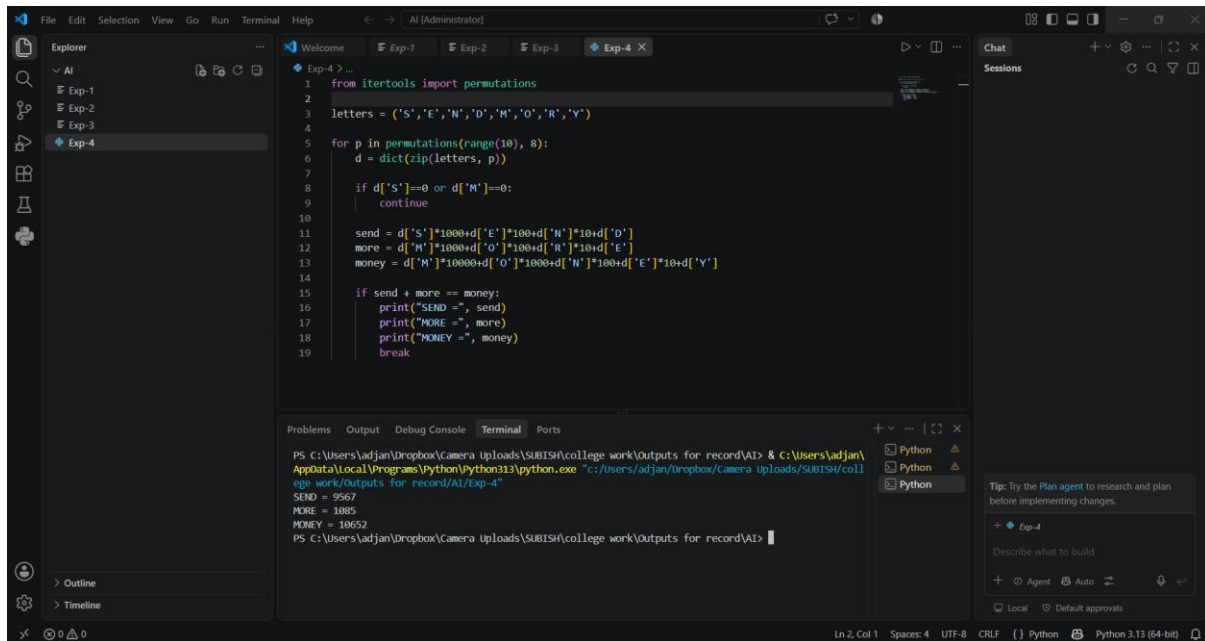
```
print("SEND =", send)

print("MORE =", more)

print("MONEY =", money)

break
```

Output:



The screenshot shows a Python IDE with a dark theme. The main editor window displays a Python script that uses the itertools.permutations module to solve the Cryptarithmic Problem. The script defines a list of letters 'S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y' and iterates through permutations of these letters. For each permutation, it assigns digits to the letters and checks if the equation SEND + MORE == MONEY is satisfied. The output of the script is displayed in the terminal window at the bottom, showing the values of SEND, MORE, and MONEY.

```
1 from itertools import permutations
2
3 letters = ('S','E','N','D','M','O','R','Y')
4
5 for p in permutations(range(10), 8):
6     d = dict(zip(letters, p))
7
8     if d['S']==0 or d['M']==0:
9         continue
10
11     send = d['S']*1000+d['E']*100+d['N']*10+d['D']
12     more = d['M']*1000+d['O']*100+d['R']*10+d['E']
13     money = d['M']*1000+d['O']*100+d['N']*100+d['E']*10+d['Y']
14
15     if send + more == money:
16         print("SEND =", send)
17         print("MORE =", more)
18         print("MONEY =", money)
19         break
```

Terminal Output:

```
PS C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI> & C:\Users\adjan\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI\Exp-4"
SEND = 9567
MORE = 1885
MONEY = 10652
PS C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI>
```

Result:

The Cryptarithmic Problem was successfully solved using the Backtracking algorithm, and a valid digit assignment satisfying the equation was obtained.

Conclusion:

The Backtracking algorithm efficiently solved the Cryptarithmic Problem by assigning unique digits to letters while satisfying all constraints.